DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
ABES ENGINEERING COLLEGE, GHAZIABAD

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: Academic Q&A System Using Retrieval-Augmented Generation (RAG)
Standard Academic Compliance: IEEE Std 830-1998

SUBMITTED BY:
Bhaskar Pandey (Roll No: 2400320100362)

Degree: Bachelor of Technology (B.Tech)
Academic Year: 2026 - 2027
Status: 3rd Year Project Specification
Date of Submission: 01-10-2026

UNDER THE GUIDANCE OF:
Mr. Ashwani Kumar
Department of Computer Science & Engg.

Project: Academic RAG System

Document Revision & Approval History

Version	Date	Description / Major Changes	Prepared By	Reviewed By
0.1	15-Sep-2026	Initial V1 scope and RAG pipeline definition	Bhaskar Pandey	Mr.Ashwani Kumar
1.0	30-Sep-2026	V1 SRS baseline for academic PDF question answering	Bhaskar Pandey	Mr.Ashwani Kumar

> Guidance Note: This SRS follows the supplied IEEE Std 830-1998 academic template and documents the current V1 notebook-based prototype. It distinguishes implemented V1 capabilities from planned future upgrades.

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional behavior, interfaces, constraints, and quality requirements of the Academic Question Answering System using Retrieval-Augmented Generation (RAG). The document provides a common reference for project development, testing, academic review, and future extension.

1.2 Scope of the System

The proposed system addresses the problem of searching and answering questions from academic study material stored in PDF documents. Instead of relying only on the language model's internal knowledge, the system retrieves semantically relevant document chunks from a vector database and supplies them as context to a generative model.

- **In-Scope Capabilities:** PDF ingestion, page-aware text extraction, recursive text chunking, cloud-based Jina embeddings, Pinecone vector storage and similarity retrieval, adaptive top-k retrieval, grounded Gemini answer generation, source/page metadata retention, and a Streamlit-based web interface for submitting academic questions, displaying answers, and viewing retrieved sources.
- **Out-of-Scope Boundaries:** Multi-user authentication, relational database management, production-grade web deployment, automatic subject classification, hybrid keyword retrieval, reranking, and large-scale multi-institution deployment are not part of the current V1 prototype.
- **Expected Benefits:** Faster retrieval of relevant academic material, grounded answers based on indexed documents, reduced dependence on manual PDF searching, and a scalable foundation for future multi-subject academic knowledge bases.

1.3 Definitions, Acronyms, and Abbreviations

Standard	SRS	Software Requirements Specification using the supplied IEEE Std 830-1998 academic structure.
RAG	Retrieval-Augmented Generation	Pipeline that retrieves relevant external context before generating an answer.
Embedding	Vector representation	Numerical representation of text used for semantic similarity search.
Vector DB	Pinecone	Cloud vector database used to store and retrieve document embeddings.
LLM	Gemini	Generative language model used to produce grounded academic answers.
API	Application Programming Interface	Interface used to communicate with Jina and Gemini cloud services.
Project Specific	Jina Embeddings	Cloud-based embedding service used to convert document chunks and user queries into 1024-dimensional vectors.

1.4 References

- **IEEE Std 830-1998:** IEEE Recommended Practice for Software Requirements Specifications.
- **Software Engineering (10th Edition):** Ian Sommerville, Pearson Education.
- Jina AI Embeddings API documentation; Pinecone vector database documentation; Google Gemini API documentation; LangChain community/PyPDF loader documentation; Python package documentation for python-dotenv and requests.

1.5 Document Overview

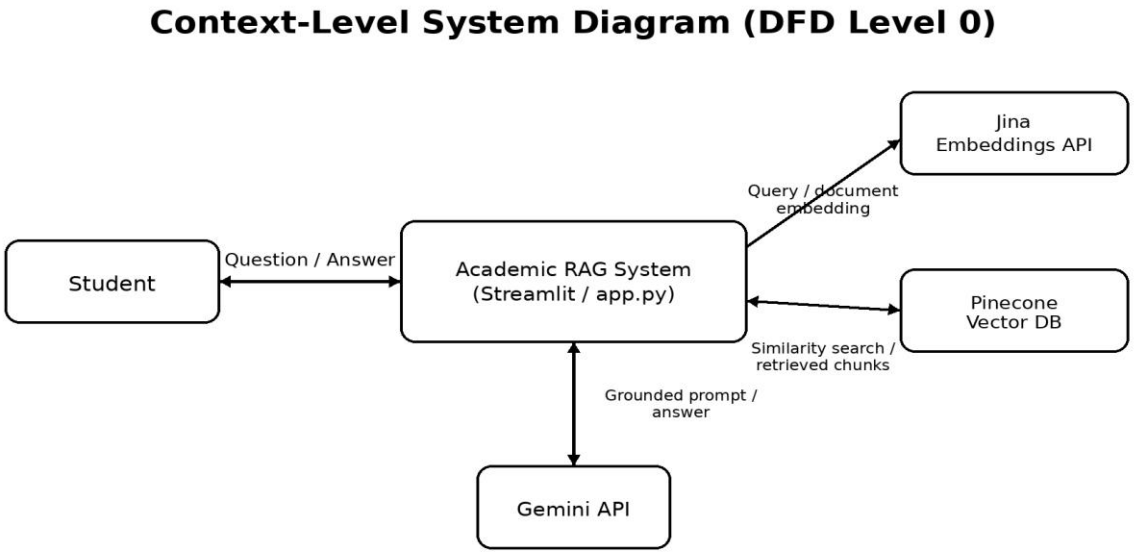
Section 2 describes the product perspective, users, operating environment, constraints, and dependencies. Section 3 specifies functional requirements for ingestion, embedding, indexing, retrieval, and answer generation. Section 4 defines software, communication, and current UI/hardware interfaces. Section 5 specifies non-functional requirements. Section 6 presents the architecture, use-case, DFD/context, and sequence models.

2. Overall Description

2.1 Product Perspective

V1 is a cloud-enabled RAG prototype consisting of a Jupyter/VS Code-based document processing and indexing workflow and a Streamlit-based web interface for user query, retrieval, and answer generation. The notebook is used for document ingestion, text processing, embedding generation, vector indexing, and experimentation, while `app.py` provides the user-facing Streamlit interface. Document text is extracted and chunked locally, while embedding generation, vector storage/retrieval, and answer generation use cloud APIs/services. The current prototype has no separate application server or relational database..

Figure 1. Context-Level System Diagram



2.2 User Classes and Characteristics

The target software accommodates distinct user personas with varying access hierarchies:

User Role / Persona	Technical Proficiency	Core Responsibilities & Rights
Student / Project User	Intermediate	Loads/maintains academic PDF corpus, submits academic questions, inspects retrieved chunks, and reviews generated answers.
Project Developer	Advanced	Configures APIs, chunking, metadata, Pinecone index, retrieval logic, prompt rules, and testing.
External Cloud Services	Service interfaces	Jina provides embeddings; Pinecone provides vector search; Gemini provides grounded response generation.

2.3 Operating Environment

In the current implementation The student/user interacts through the Streamlit web interface (app.py), while document processing and indexing are developed using Jupyter Notebook / VS Code on a Windows development environment with Python and Internet access. The prototype uses cloud APIs for embeddings and generation and Pinecone serverless infrastructure for vector storage.

- **Client/Development Environment:** Windows desktop or laptop, Jupyter Notebook or VS Code, Python runtime, and an Internet connection.
- **Cloud Services:** Jina Embeddings API, Pinecone serverless vector database, and Google Gemini API.
- **Project Storage:** Local academic PDF, notebook, and .env configuration file. No relational database is required in V1.

2.4 Design and Implementation Constraints

- **Academic Budget Constraint:** V1 is designed around cloud APIs and free/academic tiers where available; actual provider quotas and pricing may change.
- **Credential Security:** API keys must be stored in environment variables and must not be hard-coded into the notebook or committed to version control.
- **Modularity Constraint:** The current implementation uses Streamlit interface,and notebook sections are organized into ingestion, embedding, vector database, retrieval, generation, and testing stages to support later modularization.

2.5 Assumptions and Dependencies

- **Assumptions:** The academic PDFs contain extractable text, cloud APIs are reachable, API credentials are valid, and the selected embedding model produces a consistent vector dimension.
- **Dependencies:** Jina Embeddings API, Pinecone service, Gemini API, Streamlit, LangChain PDF/text utilities, Python requests, python-dotenv, and the source academic PDF.

Current V1 directory:

- Academic-RAG/
  ├── app.py
  ├── os_notes.pdf
  ├── rag_v1.ipynb
  └── .env

3. System Features and Functional Requirements

Requirements are categorized into document ingestion, embedding and indexing, retrieval, and answer-generation modules. Requirement identifiers use the FR-x.x convention.

3.1 Module 1: Document Ingestion & Chunking

Req ID	Feature Description	Input / Validation Rule	Priority
FR-1.1	PDF Document Loading	Valid academic PDF path; document must be readable by PyPDFLoader.	High
FR-1.2	Page-Aware Extraction	Extract text while retaining page metadata for later source references.	High
FR-1.3	Recursive Text Chunking	Split extracted text using configured chunk size and overlap.	High

3.2 Module 2: Embedding & Vector Indexing

Req ID	Feature Description	Input / Validation Rule	Priority
FR-2.1	Document Embedding	Send text chunks to Jina embedding API in batches.	High
FR-2.2	Vector Index Creation	Create/connect to Pinecone index with embedding dimension and cosine metric.	High
FR-2.3	Metadata Record Storage	Store chunk text, page, source, subject, vector, and unique ID.	High

3.3 Module 3: Query Retrieval & Answer Generation

Req ID	Feature Description	Input / Validation Rule	Priority
FR-3.1	Query Embedding & Retrieval	Embed the user question and retrieve top-k semantically similar chunks.	High
FR-3.2	Adaptive Retrieval Depth	Use larger top-k for broad questions and smaller top-k for focused questions.	Medium
FR-3.3	Grounded Answer Generation	Provide retrieved context to Gemini and instruct it to answer only from supplied context.	High

3.4 Module 4: Streamlit User Interface

Req ID	Feature Description	Input / Validation Rule	Priority
FR-4.1	Academic Question Input	Accept natural-language academic questions through the Streamlit chat interface.	High
FR-4.2	Display Answer	Display the generated grounded answer to the user.	High
FR-4.3	Retrieved Source Display	Allow users to inspect retrieved chunks and source page information.	High

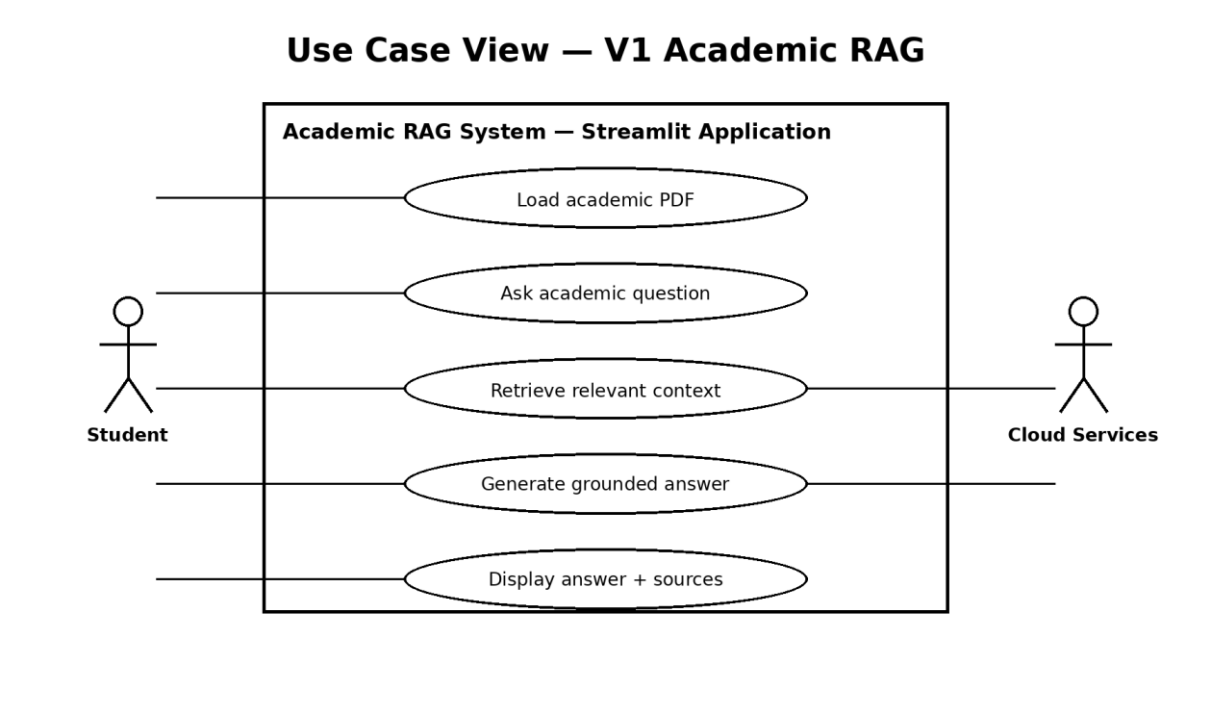
4. External Interface Requirements

4.1 User Interfaces (UI)

The system provides a **Streamlit-based web user interface** for interacting with the Academic RAG question-answering system. The interface allows users to submit natural-language academic questions and receive answers generated from the indexed academic documents.The UI provides the following components:

- **Chat Input:** Allows users to submit academic questions in natural language.
- **Conversation History:** Displays questions and generated answers during the current session.
- **Answer Display:** Presents the generated response from the Gemini model.
- **Sources:** Provides an expandable section showing retrieved chunks and their associated page numbers.
- **Error Handling:** Displays an appropriate error message when retrieval, API communication, or answer generation fails.

Figure 2. V1 User Interaction / Use Case View



4.2 Hardware Interfaces

No specialized hardware interface is required for V1. The prototype runs on a standard development computer using keyboard, mouse, display, and network connectivity.

- **Standard Development Hardware:** Laptop/desktop computer, keyboard, mouse, display, and Internet connection.
- **Embedded/IoT Varients:** Interfaces are not applicable to the current academic RAG prototype.

4.3 Software Interfaces

- **User Interface:** Streamlit-based web interface implemented in app.py
- **PDF/Text Processing:** LangChain Community PyPDFLoader and RecursiveCharacterTextSplitter. Embeddings: Jina AI API. Vector Storage/Retrieval: Pinecone. Generation: Google Gemini API. Configuration: python-dotenv.
- **Cloud Services:** Jina Embeddings API, Pinecone serverless index, and Google Gemini API. No AWS-specific service is required by the current V1 implementation.

4.4 Communication Interfaces

- **Web Traffic Protocol:** UI and Cloud API communicates through backend/API calls. API credentials are supplied through environment variables. Provider-specific request/response formats are handled by the Python client code or HTTPS requests.
- **External Cloud Utilities:** Embedding requests and API responses use JSON payloads; document content is converted to numerical vectors before storage in Pinecone.

5. Non-Functional Requirements (NFRs)

Attribute	Target Metric / Standard	Verification & Testing Method
5.1 Performance	Document embedding is batched; Pinecone retrieval uses configurable top-k. V1 is a prototype and does not claim a production latency SLA.	Record retrieval time and API response time during controlled notebook and Streamlit application tests.
5.2 Security	API keys kept outside source code in .env; no credentials should be committed to the repository.	Inspect configuration and version-control exclusions.
5.3 Reliability	Retry logic is used for Gemini generation; failure of external APIs should produce an explicit error rather than an invented answer.	Simulate transient API failures and verify retry/error behavior.
5.4 Relevance / Grounding	Generated answers must use retrieved context and should state when the answer is not found in the provided documents.	Use a manually prepared question set and inspect retrieved context and answers.

6. System Design and Analysis Models

The following models are adapted to the actual V1 RAG architecture. Because V1 uses Pinecone rather than a relational database, a conventional relational ERD is not applicable; the vector-record schema is documented instead.

Figure 3. V1 RAG Architecture

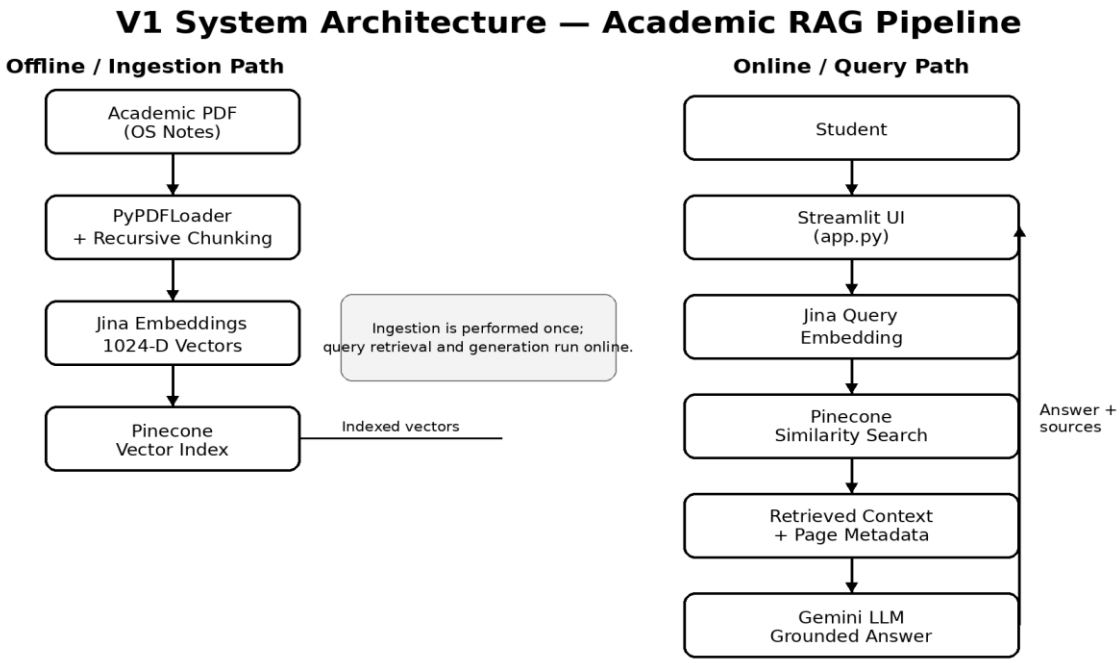
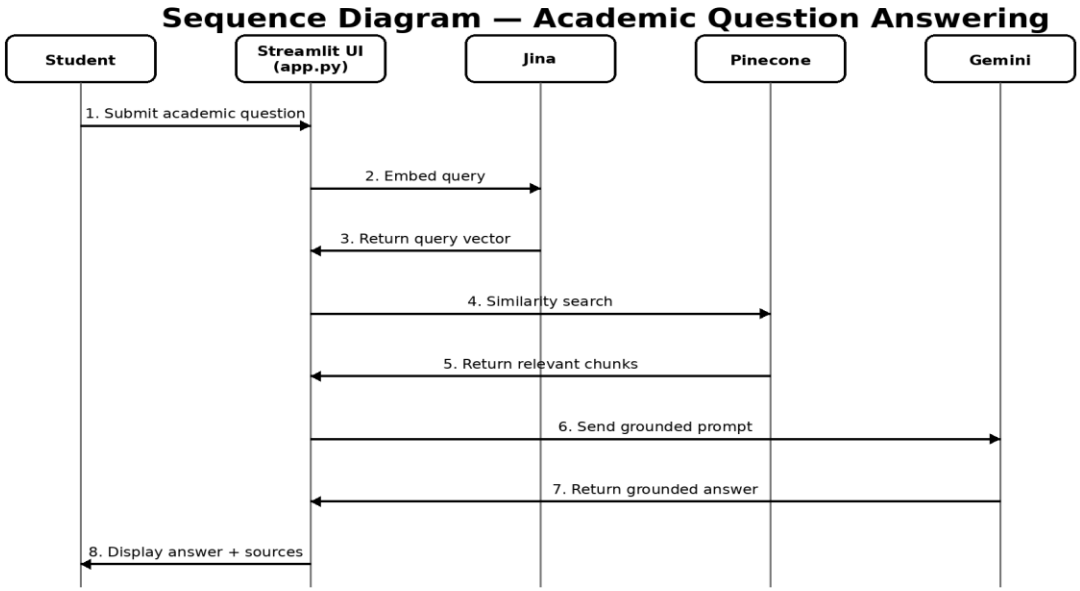


Figure 4. Question Answering Sequence



Vector Record Model (logical): id → unique chunk identifier; values → 1024-dimensional embedding vector; metadata.text → chunk text; metadata.page → source page; metadata.source → PDF filename; metadata.subject → subject label.

Diagram Type	Objective & Scope	Deliverable Format
Architecture / Context Diagram	Shows Student interaction with the RAG notebook and external Jina, Pinecone, and Gemini services.	System architecture figure.
DFD / Data Flow	Shows the flow from PDF ingestion and chunking through embeddings, vector retrieval, and generation.	Context + Level-1 flow.
Vector Record Model	Documents the logical fields stored for each Pinecone vector: id, vector, text, page, source, subject.	Schema/table representation.
Sequence Diagram	Shows the chronological question → embedding → search → context → generation → answer flow.	UML-style sequence diagram.

7. Academic Review & Sign-Off

This Software Requirements Specification documents the current V1 academic RAG prototype and is submitted for academic review and approval. Future versions may extend the corpus to multiple subjects, add subject-aware filtering, retrieval evaluation, reranking, hybrid retrieval, a production grade web interface, multi-user authentication and cloud deployment.

Project Coordinator / Guide	Internal Examiner	Head of Department (HOD)
Signature: _____ Date: _____	Signature: _____ Date: _____	Signature: _____ Date: _____